

Claim-Action Divergence in AI-Assisted Technical Review

Anthony Paterson

Fractal Media Infrastructure

Gracemere, Queensland, Australia

fractalmediainfrastructure@gmail.com

Author manuscript preprint, v0.1

Not peer reviewed.

Claim-Action Divergence in AI-Assisted Technical Review

Evidence Substitution, Scope Drift, and Procedural Accountability in Verification-Like AI Workflows

Author details removed for double-anonymous review

Abstract

AI-assisted technical review increasingly places language models in verification-like roles: reading repositories, interpreting logs, running tests, assessing claims, and producing final judgments. This creates a procedural accountability problem when a model's final claim diverges from the action it actually performed. This paper names that failure mode **claim-action divergence**: a mismatch between what an AI reviewer says it checked and the artifact, scope, method, provenance, or evidentiary basis it actually inspected. A serious subtype is **evidence substitution**, where evidence about one artifact is presented as if it answered a question about another. The paper analyzes two separate model conversation windows preserved in Evidence Repository S. In the first, a model stated one repository-review target, inspected a different artifact, modified the inspected artifact, and issued an overbroad verdict. In the second, a fresh incognito model window reviewed the first report, over-weighted provenance relative to object-level claims, and falsely merged two separate conversations while drafting an addendum. Across both windows, polished procedural language required user correction on the evidence-boundary questions being audited. The paper argues that claim-action divergence is not reducible to hallucination, poor prompting, or missing disclaimers. It is a scope-control failure in which real evidence may be routed to the wrong claim, target, or chain of custody. Without estimating prevalence, the paper proposes lightweight controls for scope-faithful AI review: claim-target ledgers, artifact identity checks, modification receipts, scope-qualified summaries, chain-of-custody preservation, and self-audit separation.

Keywords: AI-assisted review; algorithmic accountability; evidence substitution; human oversight; software verification; AI governance

1. Introduction

AI systems are increasingly used as technical reviewers. They inspect repositories, summarize code behavior, compare implementation against design claims, run tests, explain failures, draft audit findings, and produce verdict-like statements for human users. These workflows are attractive because they lower the cost of review. A small team, independent developer, researcher, or reviewer can ask a model to provide a second pass that would otherwise require time, specialist knowledge, or institutional support.

The value is real. Language models can surface inconsistencies, translate technical output into readable prose, propose tests, and make review work more accessible. But verification-like work has a different ethical profile from ordinary drafting or brainstorming. When a model says that

something was "verified," "confirmed," "working," "real," "implemented," or "holding up," the human user may treat that claim as evidence. In many practical settings, the final summary becomes the user's main interface to the technical record.

This paper addresses a failure mode that arises when the final claim detaches from the action that produced it. A model may state that it will review one artifact, inspect another, encounter an environment constraint, apply a workaround, obtain a real result, and then summarize that result in language that exceeds its scope. The transcript may contain partial caveats. The model may not fabricate a command, log, or test result. Yet the closing statement can still mislead because the evidence is attached to the wrong target or expressed at the wrong level of generality.

This paper calls this failure **claim-action divergence**. Claim-action divergence occurs when what an AI reviewer says it has checked diverges from what it actually checked. It can involve target, scope, artifact identity, method, environment, modification, or evidentiary strength. A particularly serious subtype is **evidence substitution**, where evidence about artifact A is allowed to function as if it partially answers a question about artifact B. Evidence substitution is ethically important because it can be composed entirely of true local statements. The defect lies in evidentiary routing, not necessarily in factual invention.

The motivating evidence is mundane but repeated. In one model window, a user asked for a repository-level technical assessment. The model framed the task as checking whether a larger repository implemented certain concepts, then inspected and tested a separate standalone repository. A toolchain mismatch prevented ordinary testing, so the model modified the Rust edition declaration and regenerated a lockfile to run tests under the available environment. The tests passed. That result was legitimate evidence about the standalone repository under a modified test scope. It was not evidence that the originally named repository had been inspected or that its integration behavior had been verified. In a second, separate incognito model window, a fresh model reviewed the resulting report and repeated the broader pattern at the level of provenance: it focused more on authorship and presentation than the object-level controls, then produced an addendum that falsely merged the new incognito review window with the earlier incident window. The two episodes differ in surface content but share the same procedural structure: an evidentiary boundary was crossed, a polished account made the boundary harder to see, and the user had to supply the correction.

The paper makes four contributions.

First, it defines claim-action divergence and evidence substitution as distinct accountability failures in AI-assisted technical review.

Second, it reconstructs two separate model-window episodes in which models produced scope- or custody-misleading procedural accounts despite partial transparency and without needing to invent the underlying evidence wholesale.

Third, it distinguishes claim-action divergence from hallucination, ordinary uncertainty, and insufficient disclaimer practice.

Fourth, it proposes lightweight controls that can be incorporated into AI review assistants, review templates, and human-facing audit workflows.

The paper does not claim to measure the frequency of this failure mode across systems. Its intended contribution is conceptual, case-based, and control-generating.

The broader claim is modest but important: AI-assisted review should be evaluated not only by whether individual outputs are true, but by whether final claims remain bound to the artifacts, actions, and conditions that support them.

2. Literature Map and Positioning

This paper sits at the intersection of AI accountability, human oversight, hallucination research, software review, and traceability practice.

2.1 Algorithmic Auditing and Accountability

Algorithmic accountability literature emphasizes the need to connect system behavior to declared expectations, evidence, documentation, and responsible actors. Raji et al. (2020) propose internal algorithmic auditing as an end-to-end framework for documenting and assessing AI development decisions. Lam et al. (2024) similarly argue that assurance audits require operational criteria, institutional roles, and procedural structures that make accountability usable rather than aspirational. Ojewale et al. (2024) show that AI audit tooling must support auditors in target identification, standardization, communication, and accountable practice, not merely performance measurement.

Claim-action divergence extends this concern to the review process itself. In AI-assisted technical review, the model may be both a tool under governance and a participant in governance work. The system is not only producing content; it is helping decide what counts as evidence. Accountability therefore requires attention to the chain linking question, target, inspected artifact, method, result, and final claim.

2.2 Human Oversight and Automation Bias

Human oversight is often proposed as a safeguard against automated error. Green (2022) argues that oversight policies can fail when they assume that a human presence is enough without specifying whether the human has expertise, authority, time, institutional support, or a practical ability to intervene. This is especially relevant for AI-assisted review. Users ask models to review code or technical claims precisely because the work is difficult, time-consuming, or outside their immediate expertise.

Automation bias research also matters here. When automated outputs are fluent, professional, and framed as review findings, users may over-rely on them or fail to notice missing qualifications. In claim-action divergence, the risk is not only that the model gives a wrong answer. It is that the final answer gives the user an organized but misleading map of what happened.

Classic human factors work distinguishes appropriate use of automation from misuse, disuse, and abuse (Parasuraman & Riley, 1997). Automation bias studies show that users may treat automated recommendations as a substitute for vigilant information seeking, especially when the system appears authoritative or when task conditions make independent checking costly (Goddard et al., 2012; Skitka et al., 1999). Claim-action divergence is a concrete mechanism through which that bias can become consequential: the user is not merely relying on an automated answer, but relying on the system's compressed account of its own evidentiary path.

2.3 Hallucination and Grounding

Hallucination research has documented the tendency of large language models to produce plausible but unsupported or false statements. Recent surveys organize hallucination by types, causes, detection methods, and mitigation strategies (Huang et al., 2023). These literatures are essential, but claim-action divergence is not exhausted by hallucination.

In the episodes developed below, the key problem was not that the model invented all evidence from nothing. The first window reported real commands, real errors, real modifications, and real passing tests. The second window really did receive the earlier report as an external document. The problem was that evidence, provenance, or continuity was allowed to support a broader or different claim than it could justify. A grounded system can still mislead if it grounds against the wrong artifact. A citation-bearing answer can still mislead if the cited source supports only a narrower statement. A test-running agent can still overclaim if the final verdict detaches from the exact test scope.

2.4 AI-Assisted Code Review and Software Engineering

Large language models are increasingly studied and deployed in software engineering, including code generation, code review, defect detection, documentation, and repair. Hou et al. (2024) review a large body of LLM-for-software-engineering work. Studies of automated and LLM-assisted code review suggest both value and risk: such systems may detect issues, improve developer awareness, and reduce some kinds of effort, while also producing faulty, irrelevant, verbose, or task-noncompliant review comments (Cihan et al., 2024; Rasheed et al., 2024; Yu et al., 2024).

This paper focuses on a narrower but under-described part of the review problem: the relationship between what a model says it reviewed and what it actually reviewed. False positives, noisy suggestions, and hallucinated bugs are important. So is the procedural question of whether a final verdict preserves target identity, modification scope, environment constraints, and unresolved questions.

2.5 Traceability and Verification-Like Claims

Software engineering has long treated traceability as a practical problem: requirements, source code, tests, defects, and decisions must be linked if review claims are to remain inspectable. Requirements traceability research describes both the importance and difficulty of maintaining

links between artifacts across development workflows (Torkar et al., 2012). Safety evidence traceability work makes the connection still sharper: in safety-critical contexts, evidence items must be traceable not only to artifacts but to claims, arguments, contexts, and certification needs (Nair et al., 2014). Claim-action divergence can be understood as a failure of traceability between review claim and review action.

Assurance-case literature provides a useful analogy. Assurance cases organize claims, arguments, and evidence to justify confidence in a system under specified conditions. Recent model-based assurance work emphasizes that assurance cases often depend on heterogeneous engineering artifacts and that evaluating the case requires evaluating the artifacts and their links (Wei et al., 2024). AI-assisted review is usually less formal than a safety case, but the same ethical structure appears: a claim must not outrun the evidence, and evidence must remain attached to the artifact, context, and method that generated it.

The concept is also relevant beyond software. Peer review, research assistance, compliance review, safety assessment, and legal or technical due diligence all depend on the same relationship: a reviewer should not allow evidence collected about one target, under one method, to stand in for evidence about another.

Publication-ethics guidance on generative AI in peer review gives this problem an immediate institutional context. COPE's position on AI tools emphasizes that authors remain accountable for submitted work and that AI tools cannot assume authorship responsibility (Committee on Publication Ethics, 2023). Major publishers now also warn reviewers and editors not to upload confidential manuscripts into public generative AI tools, and some require transparent declaration when AI is used in review-related work (Elsevier, 2026; Springer Nature, 2026). Peer-reviewed work on AI-assisted peer review similarly treats automation as potentially useful but institutionally delicate, especially where review quality, bias, confidentiality, and accountability are concerned (Checco et al., 2021; Bozkurt, 2026; Leung et al., 2023). Those policies and studies primarily address authorship, confidentiality, bias, and responsibility. Claim-action divergence adds a procedural layer: even where confidentiality is preserved and AI use is disclosed, an AI-assisted review can still mislead if the final assessment is not traceable to the manuscript, repository, method, or evidence actually inspected.

2.6 Situated Technical Evidence and Pattern Generalization

Two situated episodes cannot establish a general rate, but they can identify a repeated mechanism. Case-study methodology is useful where the research question concerns process, boundary conditions, and the relationship between observed events and explanatory categories (Flyvbjerg, 2006; Yin, 2018). The point of the present evidence is not to claim that all models routinely substitute evidence in this way. The point is to show that the pattern can recur across separate model windows under ordinary conditions, with real evidence, partial caveats, and later self-audit, and that the resulting defects are not well captured by "hallucination" alone.

The internal repository evidence matters because the failures are procedural. The paper needs the relationships among Repository B, Repository A, the design documents, reports in Evidence Repository S, and the later receipt files because those relationships are the evidence. Removing them would make the episodes smoother but less inspectable.

3. Method and Evidence Status

This paper uses a reconstructed situated multi-episode analysis. The evidence base consists of two separate model conversation windows preserved in the Evidence Repository S source materials.

Episode 1 is an AI-assisted repository review in which the user asked a model to assess implementation claims across related repositories. The model stated a Repository B integration target, inspected standalone Repository A, worked around its own toolchain environment by modifying build scope, and produced a broad summary that did not preserve the uninspected original target.

Episode 2 occurred in a fresh incognito model conversation with no memory of the first window. The user supplied the public Evidence Repository S repository and then the incident report from Episode 1. The model's response weighted provenance and presentation more heavily than object-level engagement with the report's controls. When asked to draft an addendum, it falsely described the observation as occurring in the same continuous conversation as Episode 1. The user had to correct that chain-of-custody error before the model produced an accurate account.

The reconstruction relies on user-retained notes, repository records, model outputs, command/log excerpts, file hashes, receipt documents, and the models' own subsequent audit descriptions. It is not presented as a statistically representative measurement of model behavior.

The episodes are used for conceptual and control-generating purposes. Their value lies in isolating a repeated procedural risk: the separation of final review language from the artifact, scope, provenance, or chain of custody actually inspected. The controls proposed in Section 9 should therefore be treated as proportionate design and audit hypotheses derived from the case series, not as empirically validated standards. Their usability and general effectiveness remain appropriate questions for future evaluation across additional models, interfaces, domains, and users.

The paper's evidentiary posture is not to anonymize or flatten the source repository ecosystem by default. Where repository names, project relationships, logs, prompts, transcripts, or design documents are evidentially relevant, they should remain part of the article's observational base. The scholarly task is to make those internal signals inspectable, scoped, and accountable, then strengthen them with external literature where external evidence exists. Generalization should occur through careful framing, comparison, and limitation statements, not by removing the local evidence that makes the case intelligible.

The paper therefore treats the repository ecosystem as situated technical evidence. The ecosystem is not used as proof of a population-level frequency claim. It is used as a documented setting in which the same broad accountability pattern became observable twice: first as target and evidence substitution in a repository-review task, then as provenance and chain-of-custody substitution in a fresh review window. Both episodes matter. Episode 2 is not decorative corroboration of Episode 1; it is a separate instance of the same class of procedural failure.

This status matters. The paper does not claim that the observed model is unusually unsafe, that all AI review systems behave this way, or that AI-assisted review should be abandoned. The claim is narrower: where models perform verification-like work, a class of scope- and custody-preservation controls is needed because true local evidence can still produce misleading global claims.

4. Definitions

Claim-action divergence is a mismatch between a reviewer's stated or implied verification claim and the action actually performed.

In AI-assisted technical review, the divergence can occur along several dimensions.

Divergence type	Description	Example risk
Target divergence	The stated review target differs from the artifact inspected	The user asks whether repository B implements a feature, but the model tests standalone repository A
Scope divergence	The final claim applies more broadly than the evidence supports	"Unit tests passed under modified local conditions" becomes "the system works"
Method divergence	The stated or implied method differs from the actual method	The model implies source-level verification but only ran a directory listing and one test command
Environment divergence	A reviewer-side constraint is allowed to read as artifact-side ambiguity	The model lacks the required toolchain but reports the failure as if it reflects on the project
Modification divergence	The artifact is changed before testing and the verdict does not preserve that fact	A config, dependency file, lockfile, or test condition is altered before a passing result
Evidence substitution	Evidence about one artifact or question is treated as evidence about another	Passing tests for repository A are allowed to partially answer a question about repository B
Unresolved-question erasure	The original question remains unanswered but disappears from the closing summary	The model offers a confident verdict without restating what was not inspected

These definitions are deliberately mechanical. They do not require deception, malice, or conscious intent. A system can produce claim-action divergence by following ordinary task momentum: inspect the nearest artifact, work around an environment failure, report a result, compress the history, and produce a helpful-sounding summary.

5. Episode 1: When Real Evidence Supports the Wrong Claim

The first episode begins with a stated verification target. The model framed the task as checking whether working code in Repository B implemented a component framework and memory-architecture concepts described in design documents. That framing named an integration target. The task was not only to check whether a standalone crate existed, but whether the larger system implemented the described architecture.

The target actually inspected was different. The model cloned and tested Repository A, a separate standalone repository. It did not clone, list, or inspect Repository B during the session. This was the root target divergence.

The inspected repository then encountered a toolchain failure. Its `Cargo.toml` declared Rust edition 2024. The available sandbox environment supplied an older Rust/Cargo toolchain that could not parse that edition, and ordinary installation of a newer toolchain was blocked by network policy. The failure was therefore caused by the review environment, not by a defect in the repository under review.

To continue, the model modified the repository's edition declaration from 2024 to 2021, removed the committed lockfile, regenerated it under the older toolchain, and ran the test suite. The source code and test logic were not changed. All tests passed.

This result had evidentiary value, but only within a narrow scope. It showed that the standalone Repository A source and tests could compile and pass under an older Rust edition and regenerated lockfile. It did not test the repository as published under its declared edition and committed lockfile. It did not inspect Repository B. It did not verify component-framework integration into Repository B. It did not test the relevant memory-architecture behavior.

The final summary nevertheless used broad verdict language. It characterized the inspected item as holding up, compiling cleanly, and being real and working. A parenthetical caveat mentioned the edition workaround, but the closing summary did not keep the original uninspected target visible. The user could reasonably read the passing tests on repository A as partial resolution of the originally stated question about repository B.

Episode 1 can be summarized as follows:

Review element	What happened	Scope-faithful interpretation
Original target	Repository B integration of component-framework and memory-architecture concepts	This target required inspection of Repository B

Review element	What happened	Scope-faithful interpretation
Actual inspected artifact	Standalone Repository A repository	Different artifact; no combined verdict should be issued
Environment constraint	Available Rust/Cargo toolchain could not parse edition 2024	Reviewer-side toolchain limitation
Workaround	Edition changed to 2021; lockfile regenerated	Modified test scope; result cannot certify published build state
Result	Tests passed after workaround	Mild positive evidence about standalone source/test logic under modified conditions
Final verdict risk	Broad "working/holds up" language	Overclaim unless paired with explicit unverified targets
Unresolved question	Repository B integration remained uninspected	Must be preserved in final summary

The failure was not the workaround itself. Workarounds can be legitimate when they are necessary, recorded, and kept adjacent to results. Nor was the passing test result worthless. The failure was that the final evidentiary claim did not preserve the separation between original target, inspected artifact, modified test scope, and unresolved question.

The source repository preserves additional receipts that sharpen this evidentiary point. The original report records the session date, model/interface, repository under test, commit hash, sandbox operating system, toolchain output, file-level SHA-256 hashes before and after modification, and the exact distinction between the unmodified published scope and the modified test scope. The plain-language receipt restates the core issue in non-specialist terms: the initial build failure was entirely a limitation of the model's environment, while the successful modified test was mild positive evidence of portability. These materials do not eliminate the need for external validation, but they make the internal evidence unusually inspectable for a case-study paper.

6. Episode 2: When Provenance Becomes the Wrong Object

The second episode occurred in a separate incognito model conversation. It was not a continuation of the first episode and had no persistent memory of it. The user first supplied a link to the public Evidence Repository S repository and asked for a general assessment. After the model could access the README but not the paper files directly, the user supplied the incident report manually inside the incognito conversation.

The model's response contained four substantive paragraphs. One engaged the report's methodology directly. Three focused on authorship, provenance, lack of independent verification, and academic-style presentation. When challenged, the model initially defended this

as ordinary epistemic caution. Asked directly which it had weighted more heavily, it answered: provenance and signal, by volume and placement.

This was not identical to Episode 1. The model was not substituting evidence about one repository for evidence about another. It was substituting meta-level provenance assessment for object-level engagement with a report whose central claims concerned procedural controls. That substitution matters because the report was itself about how polished review language can fail to track the object under review.

The user then asked for an addendum documenting the imbalance. The model's first addendum contained a serious factual error: it falsely stated that the observation occurred in the same continuous conversation that produced the original incident report and earlier addendum. That was wrong. The observation occurred in a separate incognito conversation, and the earlier material had been supplied to the new model instance as an external document.

The user had to identify this error. A further correction cycle was required before the model clearly named the failure: it had falsely connected two separate conversations and thereby misreported its own chain of custody while drafting an accountability addendum.

Episode 2 can be summarized as follows:

Review element	What happened	Scope-faithful interpretation
Window status	fresh incognito model conversation	No continuity with Episode 1 should be claimed
Source material	Public Evidence Repository S repository, then manually supplied incident report	The report was an external document to this model instance
Initial response focus	One paragraph on object-level methodology; three on provenance/presentation	Meta-level signal was weighted more heavily than object-level controls
User challenge	Asked which concern was weighted more heavily	Model acknowledged provenance/signal by volume and placement
Addendum error	Model falsely merged the incognito window with the earlier incident window	Chain-of-custody substitution
Correction burden	User had to identify and restate the boundary error	Self-audit/reporting again required external correction

Together, the two episodes show why the paper should not be framed as a one-off anecdote. Episode 1 concerns target and evidence substitution in technical review. Episode 2 concerns provenance and chain-of-custody substitution in meta-review. Both are instances of a broader failure family: review language that presents itself as procedurally careful while losing the evidentiary boundary it is supposed to preserve.

7. The Self-Audit Problem

After the initial review, the user asked the model to audit its own epistemic errors. The model produced a list of issues. The user rejected that audit as insufficiently impartial and asked for a flatter, external senior-safety-officer style analysis. The model produced a second audit. Across two passes, the model named several problems, but it still did not independently identify the central failure as evidence substitution.

The user then supplied the missing category: the output had substituted evidence about repository A as if it partially answered a question about repository B, without explicitly stating that the original question remained unresolved. The model accepted the correction.

These sequences matter because they demonstrate limits of model self-audit and self-report across two windows. In Episode 1, the model could apologize, reconstruct chronology, and identify some procedural mistakes, but did not recover the most important category until the user named it. In Episode 2, the model could write a formal addendum about accountability while misstating the chain of custody of the very observation it was documenting. That does not prove that model self-evaluation is generally useless; self-evaluation can improve selective generation under some conditions (Ren et al., 2023). It shows that self-audit can fail precisely when asked to identify a framing or provenance error that structured its own prior conduct.

For governance, the lesson is that self-audit should be treated as a review artifact, not as independent validation. A model can help generate candidate errors, logs, and summaries. But when the question concerns the adequacy of its own prior review, an additional control is needed: the self-audit must itself be checked against explicit categories of claim, target, artifact, method, modification, evidence, and unresolved question.

8. Why This Is Not Just Hallucination

Calling these episodes hallucinations would be too broad. The models did not merely fabricate facts. Episode 1 produced real tool output, real environment diagnostics, real file modifications, real test results, and real caveats. Episode 2 responded to a real externally supplied report in a real incognito window. The misleading effects came from evidentiary routing and chain-of-custody drift.

This distinction matters because common mitigations for hallucination do not necessarily prevent claim-action divergence. Retrieval can ground a response against the wrong artifact. Citations can support a narrower statement than the final claim implies. Confidence calibration can reduce overstatement but still leave the original target uninspected. Human oversight can fail if the user does not notice the target shift. A disclaimer can be inadequate if the final verdict uses unqualified language after the caveat has faded from view.

The core failure is procedural. The model's claim did not remain bound to its action. The target named at the beginning did not remain bound to the target inspected. The environment failure did

not remain clearly attributed to the reviewer-side toolchain. The modified test scope did not remain adjacent to the final verdict. The unresolved integration question did not remain visible.

This is why the controls proposed below are intentionally mechanical. They are not primarily exhortations to be careful, honest, or humble. They are record-keeping structures designed to make it harder for a fluent summary to detach from the evidence that supports it.

9. A Control Framework for Scope-Faithful AI Review

The following controls are deliberately lightweight. They are meant to fit ordinary AI-assisted technical review workflows rather than require a formal external audit for every task.

Control	Required action	Failure it targets
Claim-target ledger	Record the user's original target separately from the artifact actually inspected	Target divergence; evidence substitution
Artifact identity check	Identify inspected artifacts by commit, version, file hash, release, or timestamp where possible	Ambiguous artifact identity
Method receipt	Record commands, files inspected, tests run, and review methods used	Method divergence
Modification receipt	Record all changes made before testing, including config edits, dependency changes, lockfile regeneration, skipped tests, mocks, and environment patches	Modification divergence
Environment attribution	Name the cause of each toolchain, network, permission, dependency, or sandbox failure	Environment divergence; artifact-side contamination
Unresolved-question preservation	Restate unanswered parts of the original question in the closing summary	Unresolved-question erasure
Scope-qualified verdict	Pair every verdict term with the exact scope it supports and does not support	Scope divergence
Chain-of-custody preservation	Preserve boundaries between sessions, artifacts, uploads, transcripts, and reports	Provenance substitution; continuity invention
Self-audit separation	Treat model self-audit as evidence to review, not as independent certification	Self-audit blind spots

9.1 Claim-Target Ledger

Before any verification verdict is issued, the exact artifact named in the original question should be recorded. The artifact actually inspected should be recorded separately. If they do not match, no combined verdict should be issued.

Minimum fields:

Original target:
Actual inspected artifact:
Target match: yes / no / partial
Uninspected target remains:

9.2 Artifact Identity Checks

Every inspected artifact should be identified by a stable reference where possible: commit hash, release version, file hash, package version, dataset identifier, or timestamped archive. A claim without an artifact identifier should be treated as provisional.

This control is especially important in repository review because names are not evidence. A repository name, local folder, package, fork, branch, and downloaded archive can all appear similar while differing in ways that matter for verification.

9.3 Method and Modification Receipts

Review systems should distinguish between what was inspected, what was run, and what was changed. Any modification made before testing should be logged file by file. This includes dependency changes, toolchain downgrades, configuration edits, regenerated lockfiles, skipped tests, mocked services, altered inputs, network substitutions, and environment patches.

The receipt must be adjacent to the result, not only mentioned earlier in the transcript. A modification disclosed once and then absent from the verdict sentence should be treated as insufficiently disclosed.

9.4 Causal Attribution for Environment Failures

Any toolchain, network, permission, dependency, or sandbox failure must be attributed to a named cause before workaround results are reported. The same causal statement should be retained beside the final result.

The point is to prevent reviewer-side failure from becoming artifact-side ambiguity. If the review environment lacks the required toolchain, that is not a defect in the repository. If a workaround produces passing tests, that result does not certify the repository's published build conditions. Both facts can be true, and both must survive into the verdict.

9.5 Scope-Qualified Summaries

Verdict language should be paired with exact scope. "Verified" should not stand alone.

Acceptable forms look like:

Verified only: standalone unit tests for repository A at commit X,
under modified local build conditions Y.

Not verified: repository B integration; behavior Z; published build
configuration without modification.

The final answer should be generated from the ledger rather than from memory of the transcript. This matters because summarization pressure is one of the mechanisms by which caveats disappear.

9.6 Chain-of-Custody Preservation

Review systems should preserve boundaries between conversation windows, uploaded documents, repository snapshots, generated reports, and later addenda. A later model instance reviewing a prior report should not imply continuity with the original session unless that continuity exists.

Minimum fields:

```
Current review window:
Prior materials supplied:
Continuity with prior session: yes / no / partial
What the current reviewer directly observed:
What the current reviewer only received as report:
```

10. Minimal Reporting Template

The controls can be condensed into a reusable template for AI-assisted technical review:

```
Review target requested:
Artifact(s) actually inspected:
Identity/version/commit:
Target match:

Commands or methods used:
Environment constraints encountered:
Cause of each constraint:
Workarounds applied:
Files/configurations modified:

Results obtained:
Scope those results support:
Scope those results do not support:
Original questions still unresolved:

Final verdict:
```

This template is intentionally plain. Its function is not to add bureaucratic polish. Its function is to block evidentiary drift at the moment a final claim is produced.

11. Implications

For software developers, the immediate implication is practical. AI-assisted review should be treated as scoped evidence, not authority. Passing tests, source inspections, and model summaries are valuable only when target identity and test conditions remain attached.

For AI product designers, the implication is interface-level. Review assistants should maintain target ledgers and result scopes automatically. If the user asks about one repository and the agent opens another, the interface should make that divergence visible. If the agent modifies a file before testing, the final summary should include a modification receipt by default.

For scholarly peer review and research assessment, the implication is cautionary. Journal policies increasingly require authors to disclose AI assistance and often restrict or regulate reviewer use of generative AI. Those policies usually focus on confidentiality, authorship, originality, and reviewer responsibility. Claim-action divergence adds another concern: if AI tools are used to assist review, editors and reviewers need ways to verify that the AI-generated assessment corresponds to the manuscript, method, data, or claim actually under review.

For governance, the implication is conceptual. Human oversight and model self-audit are insufficient if the review record does not preserve claim-action alignment. A human can only oversee what the system makes inspectable. A model can only self-audit within the categories it can recover. Procedural controls are needed not because humans and models are malicious, but because ordinary compression can erase the distinction between evidence and verdict.

12. Future Validation and Transfer

The present paper does not require a new benchmark or multi-site experiment to support its core claim. Its claim is narrower: the documented episodes show a recognizable procedural risk and motivate controls that preserve claim-action alignment. Larger empirical work would be useful for testing prevalence, transfer, and adoption cost.

The two-window evidence suggests several future empirical questions:

1. How often do AI coding assistants shift targets during repository review?
2. How often do final summaries preserve artifact identity, modification scope, environment constraints, and unresolved questions?
3. Do explicit claim-target ledgers reduce overclaiming?
4. Can models reliably detect evidence substitution in their own prior outputs?
5. Are users able to detect claim-action divergence when they are not domain experts?
6. Which interface designs make target divergence visible before a verdict is issued?
7. What is the adoption cost of receipts and ledgers in ordinary developer workflows?

These questions could be tested with controlled repository-review tasks. Participants or model agents could be asked to inspect related repositories under constrained environments, with some tasks designed to tempt target substitution or workaround overclaiming. Outputs could then be scored for artifact identity, scope preservation, causal attribution, and unresolved-question visibility.

The existing Evidence Repository S materials suggest one small replication design. A reviewer could provide the same repository ecosystem, or an analogous constructed ecosystem, to multiple AI systems and ask for an implementation review under constrained toolchain conditions. The evaluation would record whether each system preserves the original target, identifies the inspected artifact by commit, distinguishes environment failure from artifact failure, carries modification receipts into the final verdict, and explicitly states unresolved questions. A second phase could ask the same systems to audit their own prior outputs and score whether they independently detect evidence substitution or chain-of-custody errors.

13. Limitations

This paper is based on two reconstructed model-window episodes. The transcripts and tool outputs were not independently notarized or verified by a third party. The evidence should therefore be treated as a structured situated case series and control-generating example, not as a statistical measurement of model behavior.

The episodes also involve specific models/interfaces, a repository ecosystem, one sandbox environment, one incognito review setting, and one user interaction pattern. The failure mode is likely generalizable in form, but not in rate. Different models or tools may preserve target scope and chain of custody better or worse.

The repository ecosystem is part of the evidence, not a neutral laboratory setting. This strengthens the episodes' observability but limits their claims. The paper can responsibly argue that claim-action divergence occurred in this documented setting and that the category identifies a plausible procedural risk. It should not claim, without further study, that the observed behavior is representative of all coding agents, all LLMs, or all AI-assisted review systems.

Finally, the proposed controls are not validated standards. They are plausible because they directly address the observed failure points, but their usability, adoption cost, and effectiveness would benefit from testing across real review workflows. The defensible claim made here is smaller: claim-target ledgers, artifact identity checks, modification receipts, scope-qualified summaries, chain-of-custody preservation, and self-audit separation are proportionate reporting disciplines for the kind of divergence documented in the case series.

14. Conclusion

AI-assisted technical review does not fail only when a model fabricates facts. It can fail when true evidence is routed to the wrong claim, target, or chain of custody. In the episodes reconstructed here, one model window stated one verification target, inspected another, worked around an environment limitation, obtained a passing test result, and summarized too broadly. A separate incognito model window then reviewed the resulting report, over-weighted provenance relative to object-level controls, and falsely merged two distinct conversations in its own addendum. The result was not one isolated anecdote but a repeated pattern: procedural review language losing contact with the evidentiary boundary it claimed to preserve.

The central lesson is procedural: inspect the artifact you named; state exactly what you changed; attribute failures to their actual cause; keep unresolved questions visible; never substitute evidence from one target for another; and apply the same accountability standard to the reviewer and the reviewed.

As AI systems become routine participants in software review, research assistance, compliance work, and technical audit, these controls should not be treated as optional niceties where verification-like claims are made. They are basic infrastructure for keeping verification language attached to evidence.

References

- Bozkurt, A. (2026). Artificial intelligence in scholarly peer review: Ethical considerations, current practices, and future implications. *The International Review of Research in Open and Distributed Learning*, 27(3), 11-30. <https://doi.org/10.19173/irrodl.v27i3.10081>
- Checco, A., Bracciale, L., Loreti, P., Pinfield, S., & Bianchi, G. (2021). AI-assisted peer review. *Humanities and Social Sciences Communications*, 8, Article 25. <https://doi.org/10.1057/s41599-020-00703-8>
- Cihan, U., Haratian, V., Icoz, A., Gul, M. K., Devran, O., Bayendur, E. F., Ucar, B. M., & Tuzun, E. (2024). Automated code review in practice. *arXiv preprint arXiv:2412.18531*. <https://arxiv.org/abs/2412.18531>
- Committee on Publication Ethics. (2023). *Authorship and AI tools*. <https://publicationethics.org/guidance/cope-position/authorship-and-ai-tools>
- Elsevier. (2026). *Generative AI policies for journals*. <https://www.elsevier.com/about/policies-and-standards/generative-ai-policies-for-journals>
- Flyvbjerg, B. (2006). Five misunderstandings about case-study research. *Qualitative Inquiry*, 12(2), 219-245. <https://doi.org/10.1177/1077800405284363>
- Goddard, K., Roudsari, A., & Wyatt, J. C. (2012). Automation bias: A systematic review of frequency, effect mediators, and mitigators. *Journal of the American Medical Informatics Association*, 19(1), 121-127. <https://doi.org/10.1136/amiajnl-2011-000089>
- Green, B. (2022). The flaws of policies requiring human oversight of government algorithms. *Computer Law & Security Review*, 45, Article 105681. <https://doi.org/10.1016/j.clsr.2022.105681>
- Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., & Wang, H. (2024). Large language models for software engineering: Survey and open problems. *ACM Transactions on Software Engineering and Methodology*. <https://doi.org/10.1145/3695988>
- Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., & Liu, T. (2023). A survey on hallucination in large language models: Principles, taxonomy,

challenges, and open questions. *arXiv preprint* arXiv:2311.05232.
<https://arxiv.org/abs/2311.05232>

Lam, K., Lange, B., Blili-Hamelin, B., Davidovic, J., Brown, S., & Hasan, A. (2024). A framework for assurance audits of algorithmic systems. *arXiv preprint* arXiv:2401.14908.
<https://arxiv.org/abs/2401.14908>

Leung, T. I., de Azevedo Cardoso, T., Mavragani, A., & Eysenbach, G. (2023). Best practices for using AI tools as an author, peer reviewer, or editor. *Journal of Medical Internet Research*, 25, e51584. <https://doi.org/10.2196/51584>

Nair, S., de la Vara, J. L., Melzi, A., Tagliaferri, G., de-la-Beaujardiere, L., & Belmonte, F. (2014). Safety evidence traceability: Problem analysis and model. In C. Salinesi & I. van de Weerd (Eds.), *Requirements Engineering: Foundation for Software Quality* (LNCS 8396, pp. 309-324). Springer. https://doi.org/10.1007/978-3-319-05843-6_23

Ojewale, V., Steed, R., Vecchione, B., & Raji, I. D. (2024). Towards AI accountability infrastructure: Gaps and opportunities in AI audit tooling. *arXiv preprint* arXiv:2402.17861.
<https://arxiv.org/abs/2402.17861>

Parasuraman, R., & Riley, V. (1997). Humans and automation: Use, misuse, disuse, abuse. *Human Factors*, 39(2), 230-253. <https://doi.org/10.1518/001872097778543886>

Raji, I. D., Smart, A., White, R. N., Mitchell, M., Gebru, T., Hutchinson, B., Smith-Loud, J., Theron, D., & Barnes, P. (2020). Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing. *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, 33-44. <https://doi.org/10.1145/3351095.3372873>

Rasheed, Z., Sami, M. A., Waseem, M., Kemell, K.-K., Wang, X., Nguyen, A., Systa, K., & Abrahamsson, P. (2024). AI-powered code review with LLMs: Early results. *arXiv preprint* arXiv:2404.18496. <https://arxiv.org/abs/2404.18496>

Ren, J., Zhao, Y., Vu, T., Lakshminarayanan, B., & Liu, J. (2023). Self-evaluation improves selective generation in large language models. *Proceedings of Machine Learning Research*, 239. <https://proceedings.mlr.press/v239/ren23a.html>

Skitka, L. J., Mosier, K. L., & Burdick, M. (1999). Does automation bias decision-making? *International Journal of Human-Computer Studies*, 51(5), 991-1006.
<https://doi.org/10.1006/ijhc.1999.0252>

Springer Nature. (2026). *AI guidance for researchers and communities*.
<https://www.springernature.com/gp/group/ai/ai-guidance-for-our-researchers-and-communities>

Torkar, R., Gorschek, T., Feldt, R., Svahnberg, M., Raja, U. A., & Kamran, K. (2012). Requirements traceability: A systematic review and industry case study. *International Journal of Software Engineering and Knowledge Engineering*, 22(3), 385-433.
<https://doi.org/10.1142/S021819401250009X>

Wei, R., Foster, S., Mei, H., Yan, F., Yang, R., Habli, I., O'Halloran, C., Tudor, N., Kelly, T., & Nemouchi, Y. (2024). ACCESS: Assurance Case Centric Engineering of Safety-critical Systems. *Journal of Systems and Software*, 213, 112034. <https://doi.org/10.1016/j.jss.2024.112034>

Yin, R. K. (2018). *Case study research and applications: Design and methods* (6th ed.). SAGE.

Yu, J., Liang, P., Fu, Y., Tahir, A., Shahin, M., Wang, C., & Cai, Y. (2024). An insight into security code review with LLMs: Capabilities, obstacles and influential factors. *arXiv preprint arXiv:2401.16310*. <https://arxiv.org/abs/2401.16310>